Ways I Use AI to Test Smarter, Not Harder

by Faezeh | Senior Test Automation Engineer

About me

- Senior Test Automation Engineer at **Nintex**, Kuala Lumpur
- 6+ years in tech, 3+ in QA automation
- Playwright, Cucumber, Cypress, Robot Framework
- Mobile automation at **Deriv**, now testing AI agent workflows



Four way I use AI

1. Generating test cases
2. Writing Playwright scripts
3. Self-healing tests
4. Flaky-test triage

Where it helps and where it needs your judgment.

1) Test cases: my set-up

- GitHub Co-pilot in VS Code
- Azure DevOps MCP tool: to read the work item and PR changes
- A SKILL.md with test generation rules

SKILL.md

Some examples of rules:

- No unit-level tests
- Assign each test to tester or developer, based on difficulty
- Decision on automation candidate
- A fixed format for test steps

and… .

Example

Feature: “Users can reset their password via an emailed link.”

```
- Skip unit-level checks (e.g. token hashing logic)
- Each scenario: title, priority, owner (QA/Dev), automate (Yes/No + reason)
- Steps: one action per line, expected result at the end
```

Scenario	Owner	Automate
Reset link works once, second use is rejected	QA	Yes
Expired link shows a clear error	QA	Yes
Reset email is delivered within SLA	Dev	No

Where it needs judgement

- Scenarios overlap, so expect to merge and trim
- Review the summary and expected result first. Check the steps in detail only for tests you'll automate

Bonus: write your own scenarios, then ask AI what's missing. It needs context, but you can quickly accept or drop each suggestion.

2) Scripts: my set-up

- Playwright for automation
- Playwright MCP: the agent sees the real page and picks real locators instead of guessing
- A SKILL.md for the output format

SKILL.md

What goes in it?

- Test title format
- Never skip or weaken an assertion just to make a test pass
- Follow our page object structure and locator conventions

and… .

Example

Bad: what AI sometimes do to get green

```
// step failed, so wrap it
try { await page.click('#submit'); } catch {}
await expect(page).toHaveURL(/dashboard/).catch(() => {});
```

Good:

```
await page.getByRole('button', { name: 'Submit' }).click();
await expect(page).toHaveURL(/dashboard/);
```

A test that can't fail is worse than no test. This is the reason the rule exists.

Where it needs judgement

- It sometimes writes complicated code where simple would do
- It saves research time on implementation details, but it still goes through normal PR review
- Instructions and skills are what moved it from hit-and-miss to reliable

3) Self-healing tests

First encounter: Testim.io

My version is human-in-the-loop: AI helps me fix the test fast, but it doesn't silently change tests on its own.

The workflow

1. Test fails. Grab the failure (error, trace, screenshot)
2. Give AI the failure plus the recent PRs and code changes
3. Ask: did the app change on purpose, or is this a real bug?
4. If intentional, update the test. If it’s a bug, raise it

Example

Failure:

```
locator('#checkout-btn') not found
```

- PR shows: the button was renamed and moved into a new “Order summary” section.
- Verdict: intentional UI change, so update the locator to `getByRole('button', { name: 'Place order' })`
- Different case: the button is missing because a feature flag broke. That’s a bug, not a test fix.

The risk

- Auto-healing can hide genuine failures. A test that heals around a real defect keeps passing while the product is broken
- Keep visibility: log every heal, and review them
- Rule: healing is only allowed for locator-level changes. A behaviour change needs a human decision

4) Flaky-test triage

Give AI the raw material:

- Failure logs and traces from several failed runs and a few passing runs
- Pass/fail history for the test
- Recent changes in the area

AI doesn't decide if a test is flaky. It reads the evidence faster than you can, and you confirm by reproducing.

A prompt that works

```
This Playwright test fails about 1 in 5 runs.

Attached: logs from 3 failed and 3 passed runs, and the test
code.

1. What differs between failed and passed runs?
2. Is this more likely timing, test data, environment, or a real
product bug?
3. What would you check to confirm each theory?
```

Watch out

- Retries hide flakiness, they won’t fix it
- If AI says ‘add a longer timeout’, ask what it’s waiting for
- A real bug can look flaky

What I have learned

- AI shifts your work from thinking up scenarios to reading and validating text. Sometimes that's more review points than writing it yourself
- Instructions and skills are what make the output trustworthy
- Use AI for documentation and drafts. Keep execution and decisions with you
- Don't be dogmatic: *"Try new tools like AI, but know their limits"*

"But test everything. Keep what is good, and stay away from everything that is evil."

None of this ships without a QA review of the AI's work.

Review the scenarios and the code, discuss with another tester. Getting lazy about reviewing is the real risk.

Demo

Questions?



Faezeh Saeedinejad

Senior Automation Test Engineer @ Nintex

Thank you for your attention.

